DOCUMENTATION · SECURITY

Security policy

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=SECURITY. This copy was made on 4 November 2026.

QueryAPIGate executes SQL on behalf of whoever can reach it, so please treat it as a sensitive piece of infrastructure. This page is an operational checklist; for the reasoning behind each control - what's actually being defended against, how it works, and where it stops - see documentation/THREAT_MODEL.md.

Reporting a vulnerability

Please **do not open a public issue**. Report it privately through GitHub's "Report a vulnerability" form on this repository, or email arcswdev@gmail.com. You can expect an acknowledgement within a few days.

Supported versions

Security fixes are made against the latest released version.

Verify a release

From 0.31, every image (`ghcr.io/anantharajuc/queryapigate`, and its `-h2` variant) and the Helm chart (`oci://ghcr.io/anantharajuc/charts/queryapigate`) is signed keyless with cosign, by digest, by this repository's release workflow - and carries SLSA provenance, an SBOM and a GitHub attestation (ADR 0018). A signature you can trust names exactly this identity:

```shell
cosign verify ghcr.io/anantharajuc/queryapigate:0.31.0 \
  --certificate-identity
https://github.com/AnanthaRajuC/QueryAPIGate/.github/workflows/docker.yml@refs/tags/v0.31.0 \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
gh attestation verify oci://ghcr.io/anantharajuc/queryapigate:0.31.0 --owner AnanthaRajuC
```

The same for the chart, `ghcr.io/anantharajuc/charts/queryapigate:0.31.0`. Anything else - another repository, another workflow, another tag, or no signature - isn't a release of this project. To have a cluster refuse it, and to read the SBOM: guide 59. Releases before 0.31 aren't signed.

SLSA level

Releases meet SLSA **Build Level 2**: each artifact is built on a GitHub-hosted runner by a workflow in this repository, and comes with provenance - which workflow, at which commit, from which tag - that the build platform signs (GitHub artifact attestations, through Sigstore), so it can't be forged without that workflow. That holds for:

- **the images and the Helm chart**, from 0.31 (`docker.yml`);
- **the sdist and wheel on PyPI**, from 0.35.1 (`release.yml`). Earlier ones carry only PyPI's own attestations (PEP 740), which say who uploaded them, not how they were built. To check one:

```shell
pip download queryapigate==0.35.1 --no-deps -d .
gh attestation verify queryapigate-0.35.1-py3-none-any.whl --owner AnanthaRajuC
```

Not Level 3. Level 3 asks that the steps doing the build can't influence the provenance. Here the attestation is made in the same job as the build, so a compromised build step - a poisoned dependency of the build, say - could in principle shape what gets attested. Building and attesting in a reusable workflow, isolated from the caller, is the documented way to Level 3; it isn't done yet.

Hardening checklist for deployments

- Set `QUERYAPIGATE_API_KEY`, and put the service behind TLS (a reverse proxy) - the key is sent in a header. It is the shared, full-access admin key; give anyone or anything that only needs to run queries a scoped key instead (`POST /api/v1/api-keys`), limited to specific connections and, separately, to read-only access.
- Give the people (and pipelines) who manage the server their own admin tokens, with the narrowest role that fits - `queryapigate admins create NAME --role owner` for the first - and then keep the shared key sealed as a break-glass key, or remove it (guide 43).
- Leave `QUERYAPIGATE_ALLOW_WRITES` unset unless you really need writes, and connect with a database account that only has the privileges the API should have. The read-only guard - including the single-statement check it relies on - is defence in depth, not a substitute for grants; it is dialect-aware (MySQL/ClickHouse honour backslash escapes in string literals by default, PostgreSQL/SQLite/H2 do not) and is exercised by a fuzz test suite (`tests/test_sql_guard_fuzz.py`), but a regex-based guard can never be a full SQL parser for every server mode.
- Keep database passwords out of the store: use `"password": "${ENV_VAR}"` references, or set `QUERYAPIGATE_SECRET_KEY` to encrypt them at rest (guide 22).
- Set `QUERYAPIGATE_SECRET_ENV_VARS` to the variables your connections reference, and `QUERYAPIGATE_METRICS_TOKEN` wherever `/metrics` can be reached from outside (guide 58).
- Run only signed releases (above), by digest.
- Do not enable `QUERYAPIGATE_DEBUG` on a reachable host.
- Set `QUERYAPIGATE_RATE_LIMIT` on anything reachable beyond a trusted network; it also throttles API key guessing.
- Behind a reverse proxy, set `QUERYAPIGATE_TRUST_PROXY` to the number of proxies so limits apply per real client - and leave it at `0` when clients connect directly, otherwise they can forge `X-Forwarded-For` to evade the limit.
- With the Rust data plane and Redis, give the data plane its own Redis user that may only count rate limits: `+evalsha +script|load +time +hmget +hset +expire` on `~qag:ratelimit:*` (guide 64). Anyone who can write those keys can reset a caller's budget.
- Only enable `QUERYAPIGATE_CORS_ORIGINS` for sites you control, and never `*` without an API key.
- Keep the query time limit (`QUERYAPIGATE_QUERY_TIMEOUT`, 30 seconds by default) and `QUERYAPIGATE_MAX_PAGE_SIZE` so one expensive request cannot monopolise the service; setting the timeout to `0` removes that protection.
- Prefer bound `:name` parameters over `{name}` text placeholders.
- Note that `GET /api/v1/connections` reveals hosts, ports, users and database names (passwords are masked).
- Plan for backing up and restoring the store - `queryapigate.db` in `QUERYAPIGATE_HOME`, or the PostgreSQL database `QUERYAPIGATE_DATABASE_URL` names (backups and restores).